

Relazione di laboratorio ASD

Tobia Viola Romanese e Mauro Brancato

2025-2026

Contents

1	Obiettivo	2
2	Organizzazione del progetto	2
2.1	Strutture Dati	2
2.2	TreeManager	2
2.3	Main	3
2.4	Grafici	3
3	Osservazioni	3

1 Obiettivo

L'obiettivo di questo progetto è quello di implementare tre tipologie di alberi binari di ricerca: **Adelson-Velsky and Landis Tree (AVL)**, **Red Black Tree (RBT)**, **Binary Search Tree (BST)** e misurare i tempi d'esecuzione dei loro metodi per verificare sperimentalmente la loro complessità teorica in termini asintotici.

2 Organizzazione del progetto

Il progetto è suddiviso nel codice in 4 parti:

- **Le strutture dati:** Abbiamo diviso in tre cartelle separate le varie strutture dati con le loro implementazioni.
- **TreeManager.py:** Abbiamo costruito un TreeManager per gestire le varie strutture dati. Questa classe ci permette di rendere il codice più leggibile, ma anche più scalabile introducendo vantaggi di OOP.
- **Main.py:** È il cuore pulsante del progetto, contiene l'inizializzazione delle varie strutture dati, la creazione della serie geometrica per un'analisi più accurata e la gestione dei vari alberi tramite il TreeManager.
- **grafici.py:** È un file che ci permette, grazie alla libreria `matplotlib.pyplot` di creare dei grafici da file `.csv`.

2.1 Strutture Dati

Nel progetto abbiamo implementato tre strutture dati per gestire gli alberi: **Adelson-Velsky and Landis Tree (AVL)**, **Red Black Tree (RBT)**, **Binary Search Tree (BST)**. La struttura dati portante del progetto è anche quella più semplice, cioè i **BST**. Le altre due classi che abbiamo implementato, infatti, ereditano le proprietà di quest'ultima; questa scelta ha una motivazione pratica (hanno dei metodi in comune che non hanno bisogno di modifiche), ma anche una motivazione teorica essendo sia **RBT** e **AVL** anch'essi alberi binari di ricerca ma con delle condizioni e specifiche ben precise per gestire l'altezza.

2.2 TreeManager

All'interno del progetto per introdurre alcuni concetti imparati nel primo semestre abbiamo deciso di implementare una classe chiamata TreeManager per sfruttare i vantaggi della OOP. La classe ha due variabili d'istanza: `keys_array` e `m`. La prima, `keys_array` è un array che contiene tutte le chiavi che da sfruttare per le misurazioni. L'array è diviso in due, nella prima parte ci sono le chiavi che sono inserite nell'albero, nella seconda metà ci sono le chiavi non ancora inserite che possono essere effettuate per la misurazione dell'inserimento. Per dividere le due parti abbiamo bisogno di un indice che scorre man mano che riempiamo l'albero inserendo le chiavi ed è qui che entra in gioco la seconda variabile d'istanza `m`, un intero che separa le chiavi esterne da quelle già inserite nell'albero. In questa classe abbiamo costruito due metodi principali: `create_tree` e `measure_insert`.

Il metodo `create_tree` riceve in input un albero (`tree`) e il suo scopo è quello di riempire la struttura dati in input con le chiavi all'interno della porzione di array `keys_array[m : n]` con $n = \text{length}(\text{keys_array})$. Queste chiavi vengono selezionate in modo casuale così da garantire una distribuzione casuale uniforme. Una volta selezionata la chiave, viene creato il nodo e viene inserito nell'albero. Ora la chiave selezionata viene spostata all'interno della porzione dell'array `keys_array[0 : m - 1]` perché ora la chiave fa parte dell'albero e di conseguenza aumentiamo anche l'indice `m` di uno.

Il metodo `measure_insert` riceve in input un albero (`tree`) e un numero di iterazioni (`batch_size`) che corrisponde al numero di tempi raccolti per una certa albero di dimensione `n`. All'inizio del metodo viene creato il vettore che conterrà tutti i tempi misurati, successivamente c'è il ciclo for che costituisce il corpo del metodo. All'interno del ciclo viene scelta la chiave da inserire e viene creato il nodo corrispondente da inserire nell'albero, viene fatto partire il timer, si effettua l'inserimento e il timer viene fermato. Il tempo calcolato viene inserito nel vettore dei tempi e il vettore `keys_array` viene sistemato a dovere scambiando la chiave appena inserita nell'albero nella prima parte dell'array. Sempre all'interno del ciclo per ottenere delle misurazioni corrette per una dimensione fissa `n` dobbiamo rimuovere una chiave in modo casuale dall'albero. Selezioniamo questa chiave nella prima parte dell'array e cerchiamo il nodo corrispondente da estrarre, rimuoviamo il nodo e decrementiamo l'indice `m` scambiando anche le chiavi nell'array. Alla fine del ciclo for il metodo restituisce la mediana dei tempi.

2.3 Main

Questo codice è il cuore pulsante del progetto. Nella prima parte vengono inizializzate tutte le strutture dati che ci servono per creare la progressione geometrica che verrà utilizzata per definire la quantità delle chiavi (e quindi anche degli alberi). Successivamente nel codice è presente un ciclo for al cui interno creiamo la dimensione dell'albero e il **TreeManager**. Dopo l'inizializzazione del manager vengono inizializzate e misurati i tempi, inserendoli anche nel vettore dei risultati, **una alla volta** per mantenere la sincronizzazione tra l'array delle chiavi nel TreeManager e nell'albero. Completato questo ciclo vengono eseguite una serie di istruzioni che permettono la creazione di un file `.csv` contenente una tabella di tutti i tempi misurati delle tre strutture dati e la dimensione sui cui si è effettuata la misurazione.

2.4 Grafici

Questo codice è essenziale per generare le immagini dei grafici. Prima il codice legge il file `misurazioni_alberi.csv` generato dal `Main.py` e prende tutti i tempi che, grazie alla libreria `matplotlib.pyplot`, vengono visualizzati con tre colori diversi per distinguere le tre strutture dati. Sull'asse delle ascisse troviamo la dimensione degli alberi in scala logaritmica e sull'asse delle ordinate troviamo le misurazioni dei tempi.

3 Osservazioni